Deploying Agentic AI Leads on Coolify

A step-by-step guide to deploy the Agentic AI Leads CRM on **Coolify**. Follow it top to bottom. Where you need to fill something in, it looks like `<THIS>` — replace the whole thing, including the angle brackets.

What you're setting up: the CRM runs as a Docker container that Coolify builds from the Git repo. Two people log in with a username/password. The data-gathering "engines" run automatically on a schedule. All data is stored on a **persistent volume** so it survives redeploys.

The repo already contains everything Coolify needs — a `Dockerfile`, an entrypoint script, and a built-in login page. You mostly click through the Coolify UI and set a few environment variables.

Estimated time: **20–30 minutes.**

---

0. Before you start — what you need

1. **The Git repository** (a private GitHub repo) and a way for Coolify to pull it (a connected GitHub App or a deploy key). Get the repo URL from Haron.
2. **The secret keys** for `.env` values — at minimum the **Apollo API key**, plus any email-delivery keys. These are NOT in the repo; Haron sends them separately.
3. **The two login credentials** you'll use — decide a username + password for each of you (e.g. `haron` / a strong password, and one for your cofounder).
4. A running **Coolify instance** with a server already connected to it. (If you don't have Coolify installed yet, install it first at https://coolify.io — one command on a fresh server.)

---

1. Create the application in Coolify

1. In Coolify, open your **Project** → **Environment** → **+ New** → **Application**.
2. Choose **Public Repository** or **Private Repository (with GitHub App / Deploy Key)** and paste the repo URL. Select the branch (usually `main`).
3. For **Build Pack**, choose **Dockerfile**. Coolify will detect the `Dockerfile` in the repo automatically.
4. Set the **Port** to `8501` (this is the port the app listens on).
5. Click **Continue** / **Create** — but **do not deploy yet**. Do Steps 2 and 3 first, otherwise the first run will start without credentials or a data volume.

---

2. Add the environment variables

Open the application's **Environment Variables** tab and add the following. Mark them all as **secret** / **build-time not required** (they're runtime values).

Login credentials (choose ONE style):

- Simplest, supports both of you — one variable:

Name	Value
`APP_USERS`	`<you>:<your-password>,<cofounder>:<their-password>`

Example: `APP_USERS = haron:Sxr9-tuna-42,sam:Lp2-mango-88`

- Or, for a single shared login, use two variables instead:

Name	Value
`APP_USERNAME`	`<username>`
`APP_PASSWORD`	`<password>`

API / **delivery keys** (whatever Haron gave you):

Name	Value
`APOLLO_API_KEY`	`<apollo-key>`
(any delivery keys, e.g.) `INSTANTLY_API_KEY`	`<key>`

You do **not** need to set `AAL_DATA_DIR` — the image already sets it to `/data` (the volume you add next).

> Passwords here are stored as plain text in Coolify's environment. That's fine for this setup — they're never committed to Git. Use strong, unique passwords since the app will be reachable on the internet.

3. Add the persistent data volume — do not skip this

This is the most important step. Without it, **every lead stage, sweep result, and enrichment is wiped on each redeploy.**

1. Open the application's **Storages** (or **Persistent Storage**) tab.
2. Add a new volume: - **Name:** `aal-data` - **Mount Path (in container):** `/data`
3. Save.

That's it — the app and all engines read and write under `/data`, and the entrypoint seeds it with the repo's starter `watchlist.json` and client configs on first boot.

---

4. Deploy

Click **Deploy**. Coolify builds the image and starts the container. Watch the deploy logs — near the end you should see a line like:

```
starting Streamlit on 0.0.0.0:8501 (data dir: /data)
```

and no errors. Once it's healthy, open the app's URL (Coolify shows it at the top of the application page). You should see the **"Agentic AI Leads — Sign in"** page. Log in with one of the credentials from Step 2.

> **Accessing it / domains.** By default Coolify gives the app an address you can use immediately. HTTPS isn't required for now, as agreed. When you do want a proper `https://` address later: attach a domain to the application in Coolify's **Domains** field and Coolify's built-in proxy issues a free TLS certificate automatically — no app changes needed.

---

5. Schedule the engines (automatic data updates)

The engines gather leads and signals on a schedule. Coolify runs them inside the container via **Scheduled Tasks**.

> **One-time first run (do this once, by hand, before scheduling).** The very first signal sweep must record a baseline quietly. In the Coolify application, open the **Terminal** / **Execute Command** (or use the Scheduled Task "Run Now" after creating it) and run:
>
> `python signal_engine.py --baseline`
>
> If you'll use the second ("prospector") pipeline, Haron should also run `python prospector_engine.py --probe` once with the real Apollo key before you schedule it. If unsure, leave the prospector task out for now.

Open the application's **Scheduled Tasks** tab and add these (Command + Frequency in cron format). Each runs inside the app container, so it shares the `/data` volume and the environment variables automatically.

Name	Command	Frequency (cron)
Nightly signal sweep	`python signal_engine.py`	`0 3 * * *`
Weekly watchlist growth	`python discovery_engine.py --hn 3`	`0 4 * * 1`

Name	Command	Frequency (cron)
Weekly contact enrichment	`python enrichment_engine.py`	`30 4 * * 1`
Reply sync (3×/day)	`python delivery.py --sync-replies`	`0 9,13,17 * * *`

The app already prevents two engines from running at once (a lock file on the volume), so a scheduled sweep won't collide with someone clicking "run" inside the app.

---

6. Set up backups of the data volume

Since the `/data` volume is now your database, back it up. The simplest reliable approach is to snapshot the volume from the **host server** that runs Coolify.

SSH into that server and add a nightly cron job (`crontab -e`):

```
# Back up the Agentic AI Leads data volume nightly at 02:30, keep 30 days
30 2 * * *  tar -czf /root/aal-backups/aal-$(date +\%Y-\%m-\%d).tar.gz -C /var/lib/docker/volumes ${D
```

If you're not comfortable with that one-liner, the dependable manual version is: find the volume with `docker volume ls | grep aal-data`, then `docker run --rm -v <volume-name>:/data -v /root/aal-backups:/backup alpine tar -czf /backup/aal-$(date +%F).tar.gz -C /data .` run on a schedule. For real safety, copy the backups off the server periodically (a backup on the same machine won't survive a disk loss).

---

7. Final verification checklist

- ☐ The deploy log ends with `starting Streamlit ...` and no errors.
- ☐ Opening the app URL shows the **Sign in** page.
- ☐ Both credentials log in; a wrong password is rejected.
- ☐ The **Storages** tab shows the `aal-data` volume mounted at `/data`.
- ☐ `python signal_engine.py --baseline` has been run once.
- ☐ The **Scheduled Tasks** tab lists the engine jobs.
- ☐ After a redeploy, your data is still there (change a lead's stage, redeploy, confirm it persisted) — this proves the volume works.

---

8. Everyday operations

Update to new code: push to the Git branch. If **Auto Deploy** is on in Coolify, it redeploys automatically; otherwise click **Deploy**.

Change a password / API key: edit the Environment Variable in Coolify, then **Redeploy** (or Restart) so the container picks it up.

Run an engine right now: Scheduled Tasks → the task → **Run Now**, or use the application **Terminal** and run the command directly.

View logs: the application's **Logs** tab (app) and each Scheduled Task's run history (engines).

---

9. Troubleshooting

The Sign-in page says "Login isn't configured." The credential env vars aren't set. Add `APP_USERS` (or `APP_USERNAME` + `APP_PASSWORD`) in Environment Variables and redeploy.

Data disappears after every deploy. The persistent volume is missing or mounted at the wrong path. It must be mounted at exactly `/data` (Step 3). Redeploy after adding it.

The app builds but the page won't load / keeps reconnecting. Confirm the Coolify Port is set to `8501`. If you put the app behind a custom domain/proxy, make sure websockets are allowed (Coolify's built-in proxy handles this automatically).

An engine fails with an SSL / certificate error. The image installs the `truststore` package the engines rely on; this normally just works in the container. Re-run the task; if it persists, check the Apollo key is set.

An engine says it's locked / already running. Only one engine runs at a time by design. Wait for the current run to finish. A stale lock file would be at `/data/.engine.lock` on the volume; remove it only if you're certain nothing is running.

Appendix — how the pieces fit

- **`Dockerfile`** builds the image and sets `AAL_DATA_DIR=/data`.
- **`docker-entrypoint.sh`** seeds `/data` from the repo's committed `watchlist.json` and `configs/` on first boot (never overwriting existing data), then starts Streamlit on `0.0.0.0:$PORT` (default 8501).
- **`auth.py`** is the login gate; it reads `APP_USERS` / `APP_USERNAME` / `APP_PASSWORD` from the environment. Login lasts while the browser tab is open.
- **The engines** all read/write under `/data` (via `AAL_DATA_DIR`), so the app and the scheduled tasks share one consistent data set on the volume.